

| Registros De Ativação

João Eduardo Montandon

DCC024

Qual a diferença entre os programas abaixo?

```
int f() {  
    int a = 0;  
    return a;  
}
```

```
fun f() = let val a = 0 in a end;
```

| Introdução

- Variáveis são vinculadas a valores
- Esses valores precisam ser guardados em algum lugar
- Portanto, variáveis precisam também **ser vinculadas a posições de memória**

Como? 🤔

| Ativações De Função

- O *tempo de vida* da execução de uma função começa na sua chamada, e termina em seu retorno.
- Esse período é conhecido como *ativação da função*.

| Variáveis Dinâmicas

- Quando cada ativação tem sua própria vinculação de uma variável a um local de memória, *essa variável é específica da ativação*.
- Essas variáveis são conhecidas como variáveis dinâmicas ou automáticas.

```
fun days2ms days =  
  let  
    val hrs = days * 24.0  
    val mins = hrs * 60.0  
    val secs = mins * 60.0  
  in  
    secs * 1000.0  
  end;
```

Qual o tempo de vida das variáveis `days`, `hrs`, `mins` e `secs`? 🤔

| Ativação De Blocos

- Blocos também podem "ativar" a vinculação de variáveis definidas dentro deles.
- Neste caso, o tempo de vida das variáveis ficam atreladas ao tempo de ativação do bloco.

```
fun fact n =  
  if (n = 0) then 1  
  else let val b = fact (n - 1) in n * b end;
```

Qual o tempo de vida das variáveis `n` e `b`? 🤔

O tempo de vida de uma variável pode ser maior que o tempo de vida da ativação da função que a alocou? 😬 😬

| Variáveis Estáticas

- Variáveis que **não estão vinculadas** a ativação que as criou.
 - Sua existência se estende entre diferentes chamadas da função.
-

```
#include <stdio.h>
int f() {
    static int count = 0;
    count++;
    return count;
}
int main() {
    printf("%d\n", f());
    printf("%d\n", f());
    printf("%d\n", f());
    return 0;
}
```

Para pensar...

- Qual o tempo de vida de `count`?
 - Qual o escopo de `count`?
-

| Registros De Ativação

- Toda *chamada de função* deve alocar os dados necessários para sua ativação.
 - Endereço referente ao local de retorno da função.
 - Endereço de seus parâmetros.
 - Referência para as variáveis criadas durante sua ativação.
 - Essas informações são armazenadas no **Registro de Ativação (RA)**.
-

As linguagens implementam diferentes formas de gerenciar RAs.

1. Alocação Estática
 2. Alocação de Pilha
 3. Funções Aninhadas
 4. Funções Como Parâmetro
 5. Funções de Longa Duração
-

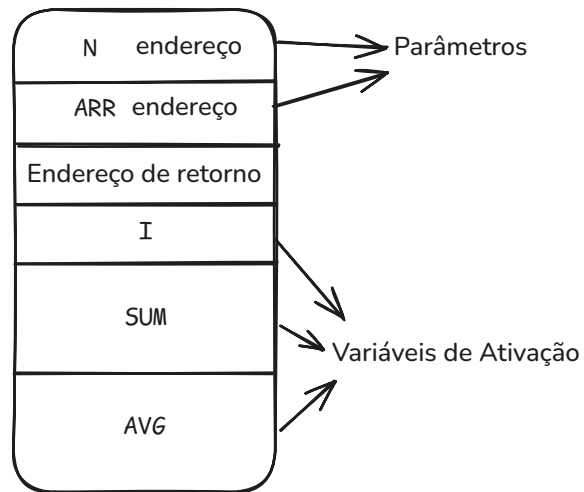
| Alocação Estática

- Alocar um RA por função, *estaticamente*.
 - Cada função terá acesso ao seu registro de ativação antes da execução do programa.
 - Exemplos: COBOL, FORTRAN
-

```

FUNCTION AVG (ARR, N)
DIMENSION ARR (N)
SUM = 0.0
DO 100 I = 1, N
SUM = SUM +
ARR(I)
100 CONTINUE
AVG = SUM / FLOAT(N)
RETURN
END

```



Quais as vantagens?

- Simplicidade de implementação: apenas um RA por definição de função.
- Alto desempenho: o RA é definido em tempo de compilação

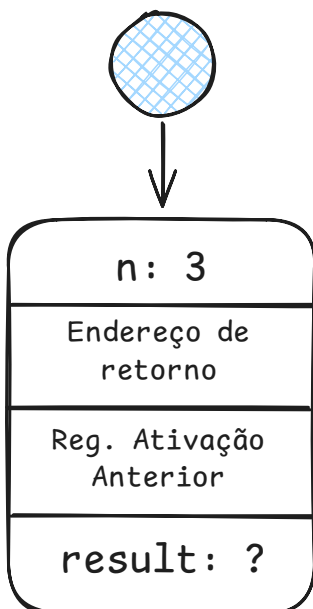
Quais as limitações?

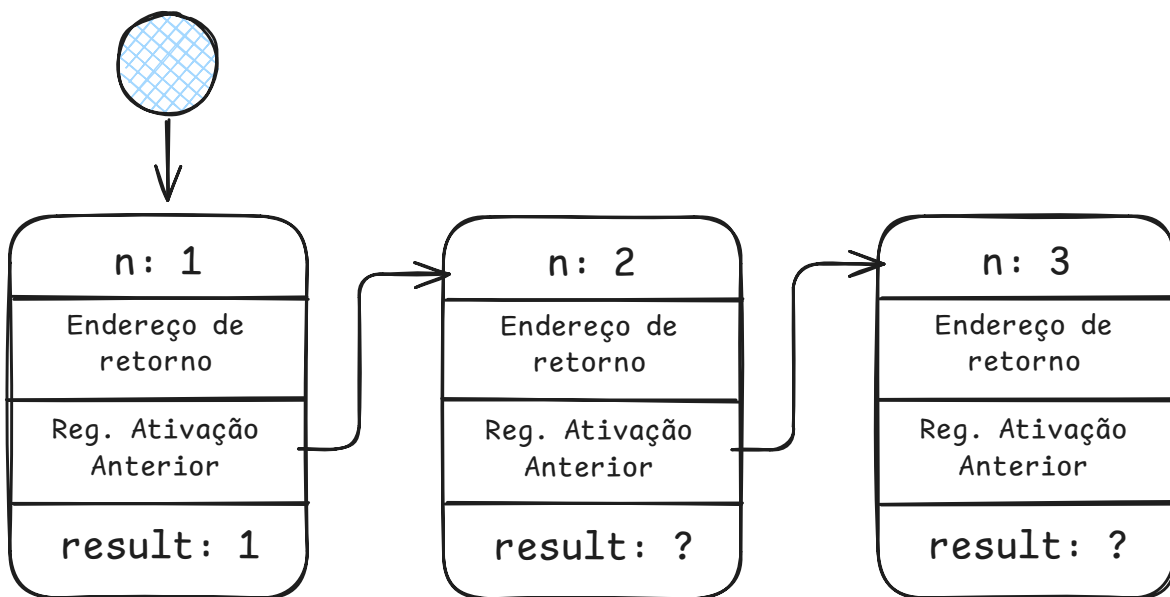
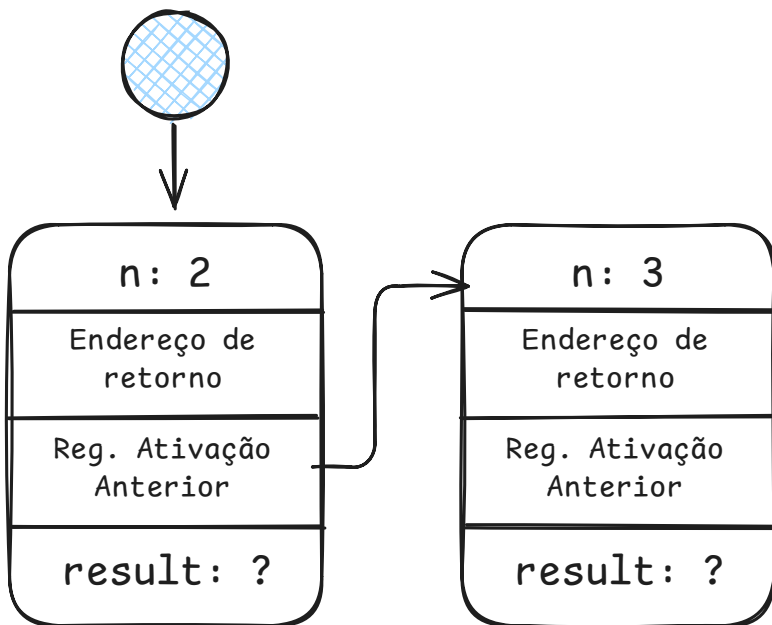
- As chamadas a uma mesma função utilizam o mesmo RA.
- Que limitação isso traz para uma linguagem? 🤔

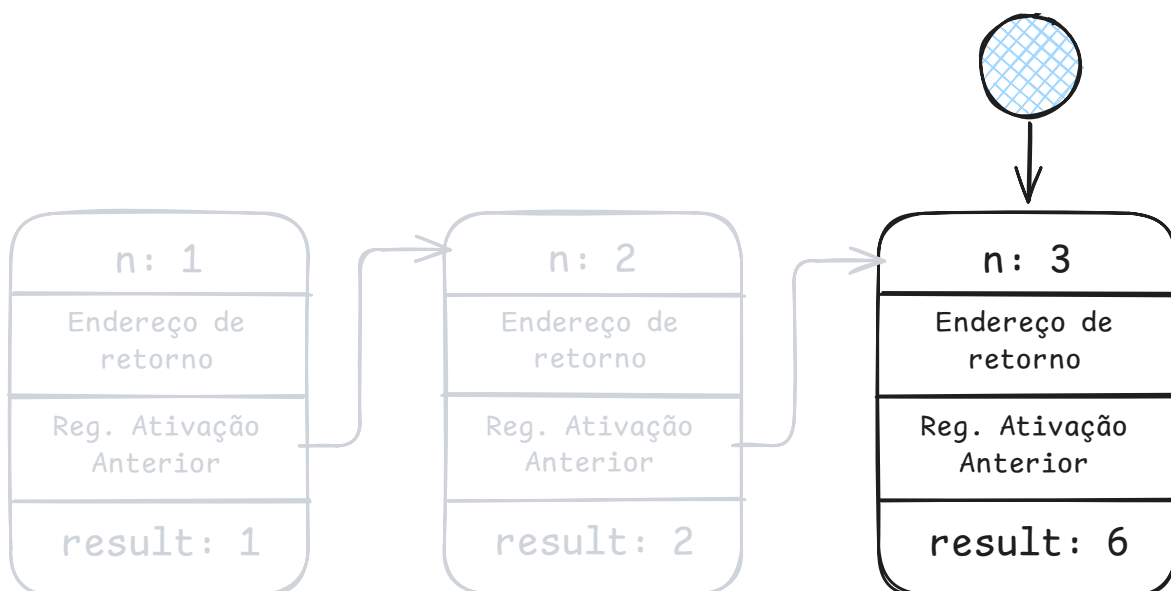
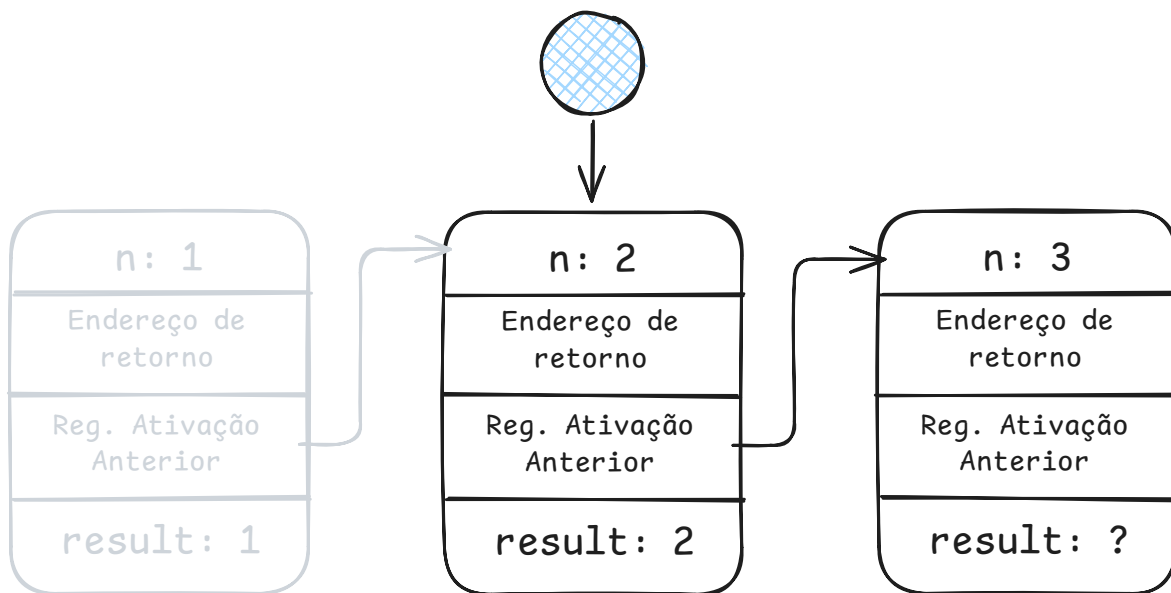
| Alocação De Pilha

- Um novo RA é criado quando uma função é chamada.
- RAs agora estão atrelados a chamada da função.
- Por isso, são alocados dinamicamente em *runtime*.
- Controle é feito via *stack frame*.

```
int fact(n) {  
    int result;  
    if(n < 2) result = 1;  
    else result = n * fact(n-1);  
    return result;  
}  
int main() {  
    fact(3);  
}
```



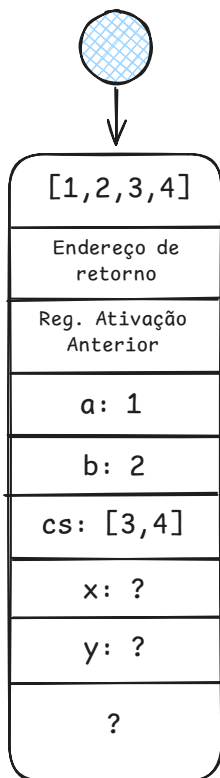


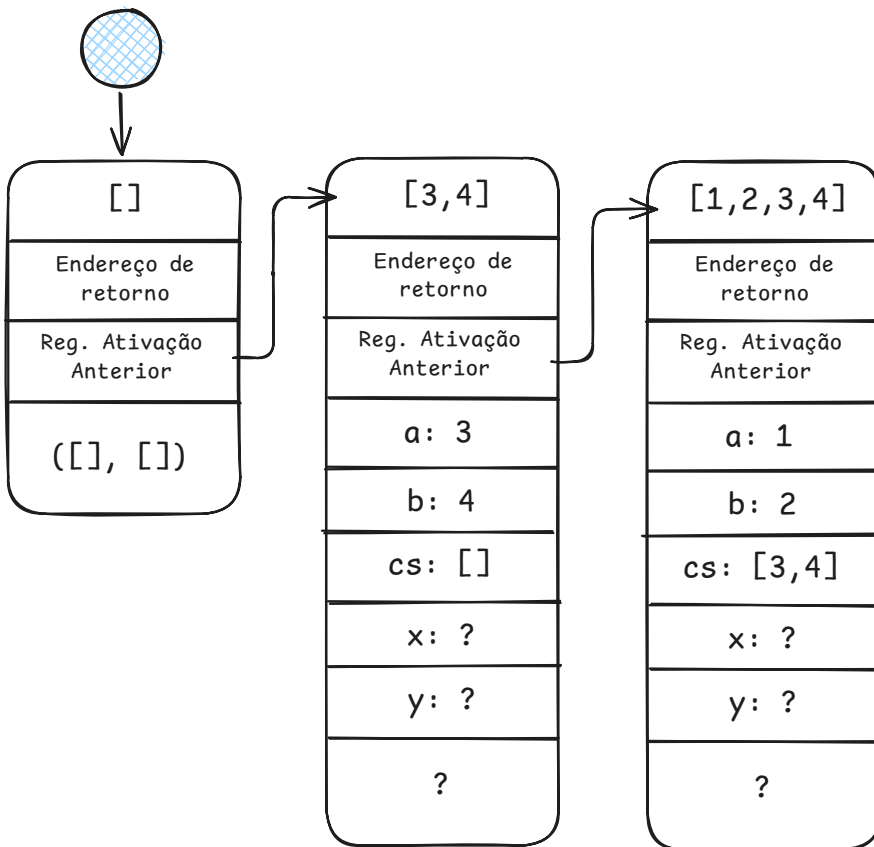
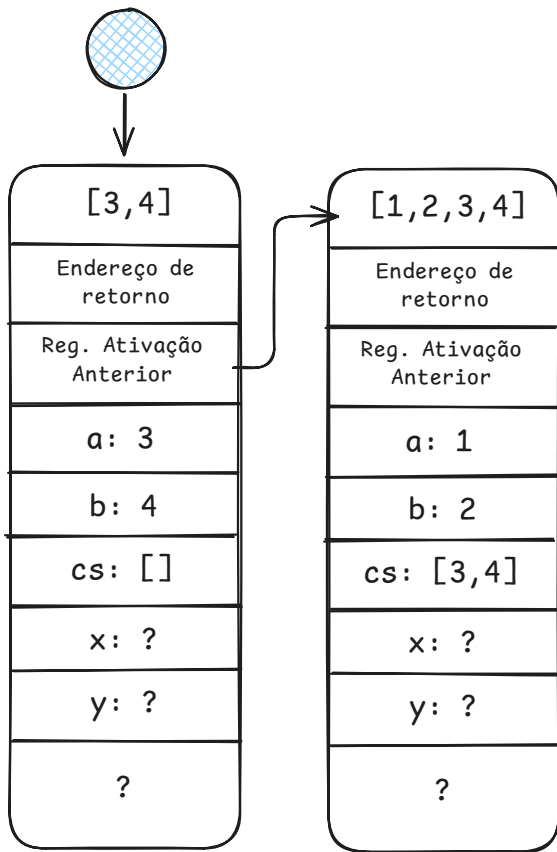


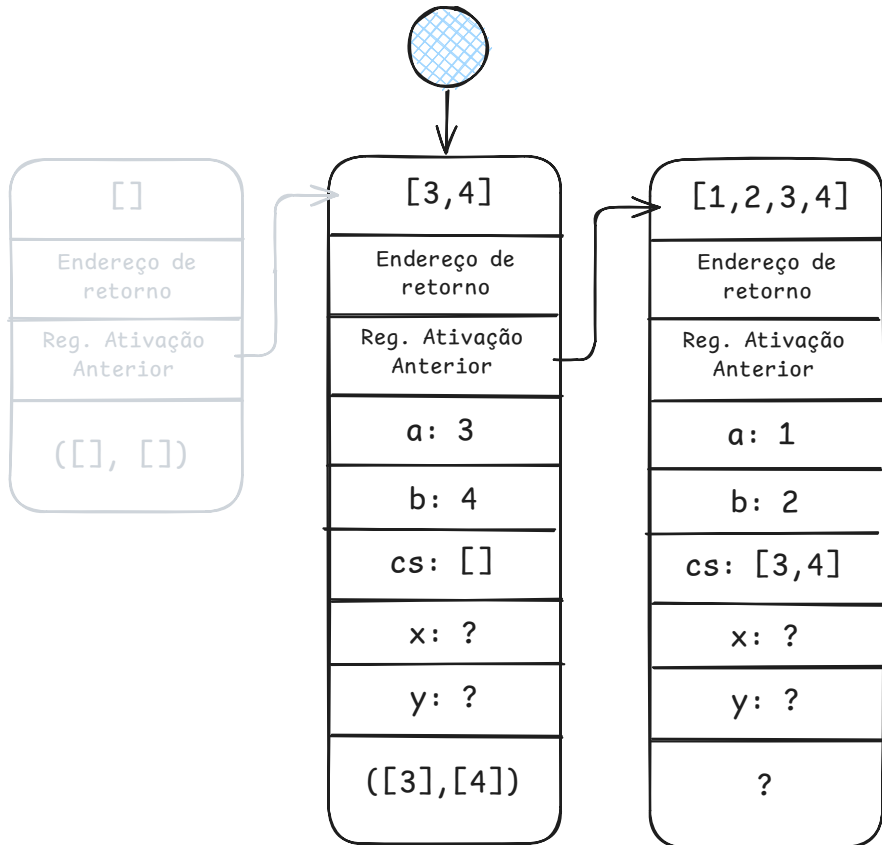
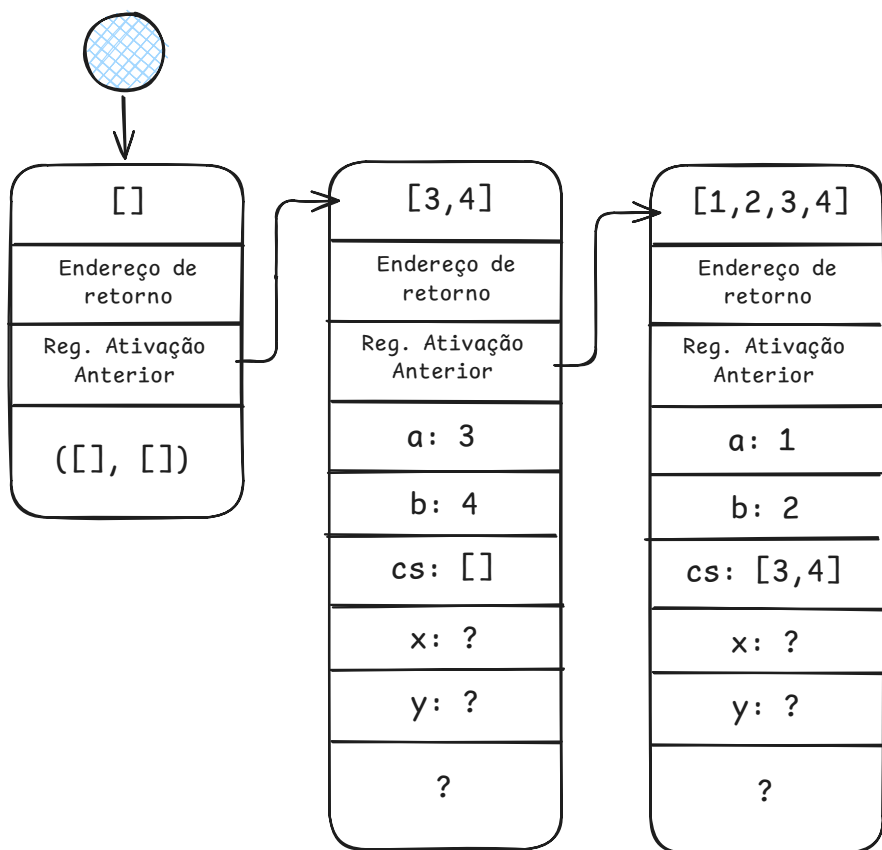
- *Stack frame* controla a ordem e as informações presentes em cada RA.
- RAs agora armazenam um ponteiro para o RA da ativação que os chamaram.
- Uma vez processado, o RA é desalocado.

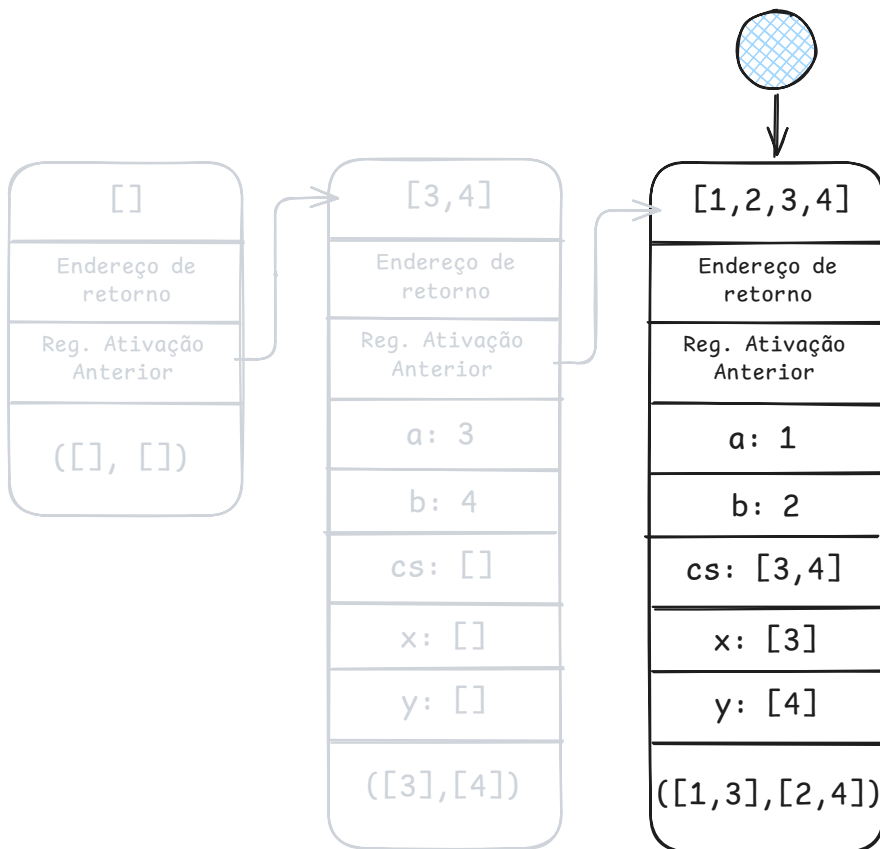
```
fun halve nil = (nil, nil)
| halve [a] = ([a], nil)
```

```
|   halve (a::b::cs) =  
      let  
      in   val (x, y) = halve cs  
      (a::x, b::y)  
    end;  
halve [1,2,3,4];
```





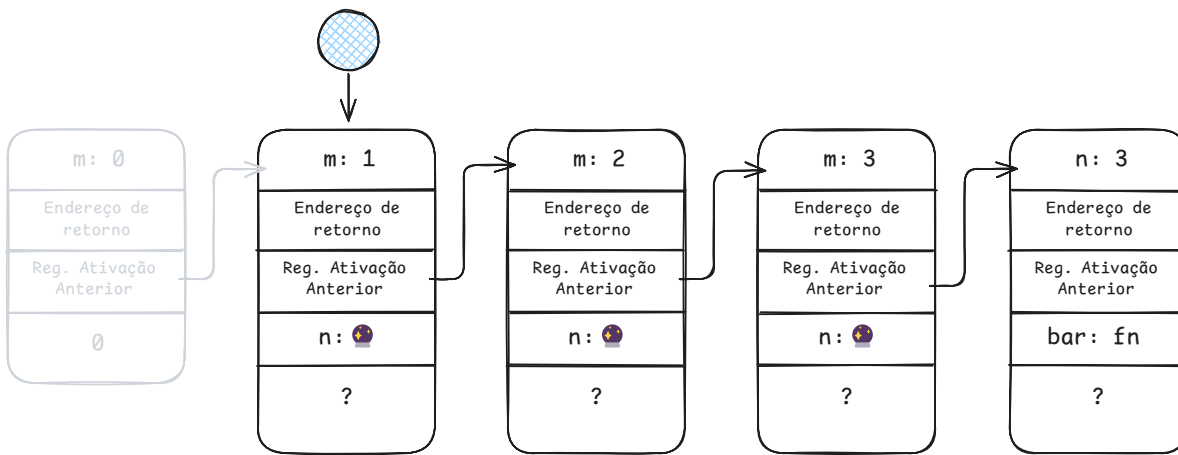




| Funções Aninhadas

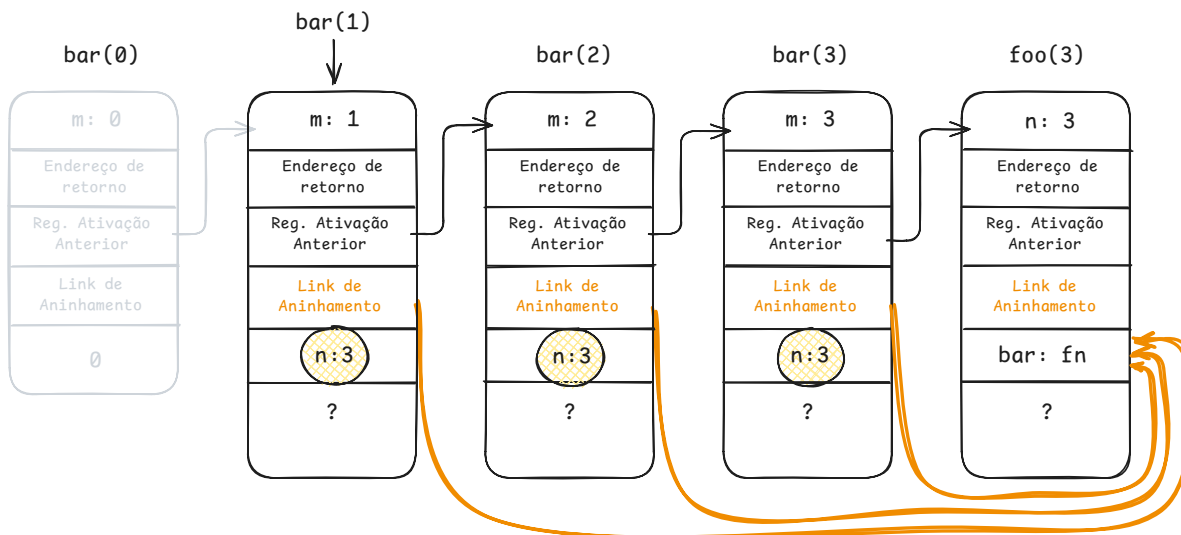
O que será impresso nesse programa? 🤔

```
fun foo n =
  let
    fun bar 0 = []
      | bar m = n :: bar (m - 1)
    in
      bar n
    end;
  foo 3;
```



Onde está n? 🤔

- Um RA já possui referência a quem o chamou (caller).
- Mas precisamos de uma referência do RA que **definiu a função ativada**.
- **Link de Aninhamento:** Referência que aponta para o RA com a definição mais recente da função ativada.



Algoritmo

- função não é aninhada → `null`.

- Chamada da função aninhada a partir da função externa ➡ RA do *caller*.
- Chamada da função aninhada a partir da função interna ➡ Link de Aninhamento do *caller*.

| Aninhamento Multi-Nível

```
fun f1 n1 =  
  let  
    fun f2 n2 = let fun f3 n3 = n1 + n2 + n3 in f3 3  
  end  
  in  
    f2 5  
  end
```

De onde vem `n1`, `n2`, e `n3` 🤔?

- Processo de busca é recursivo.
- Referências locais: *RA*.
- Referências a **um** aninhamento de distância: $LA \rightarrow RA$.
- Referências a **dois** aninhamentos de distância: $LA \rightarrow LA \rightarrow RA$.
- Referências a **n** aninhamentos de distância: $LA \xrightarrow{n} RA$.

| Para Pensar (1)

Consegue imaginar outras soluções para este caso? 🤔

| Para Pensar (2)

Essa referência é possível pois tanto a definição quanto a referência estão **no mesmo escopo**.

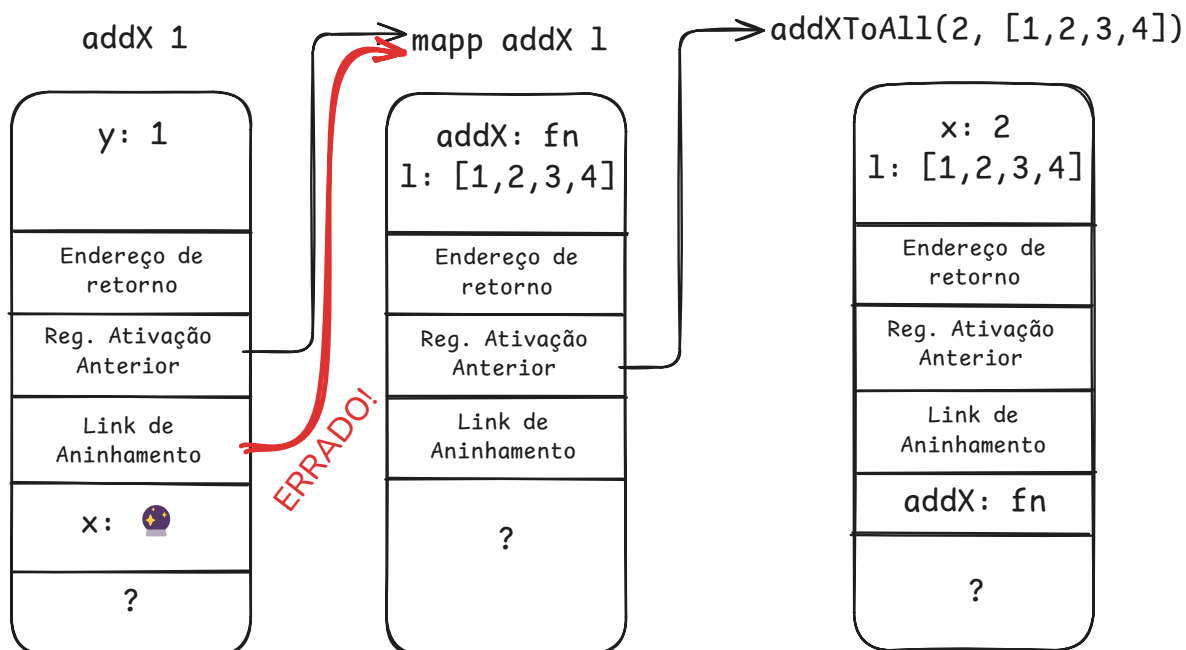
E se o escopo entre as duas for diferente? 🤔

Isso é possível? 🤔

| Funções Como Parâmetro

```
fun addXToAll (x, l) =  
  let  
    fun addX y = x + y  
  in  
    map addX l  
  end;  
addXToAll (2, [1,2,3,4]);
```

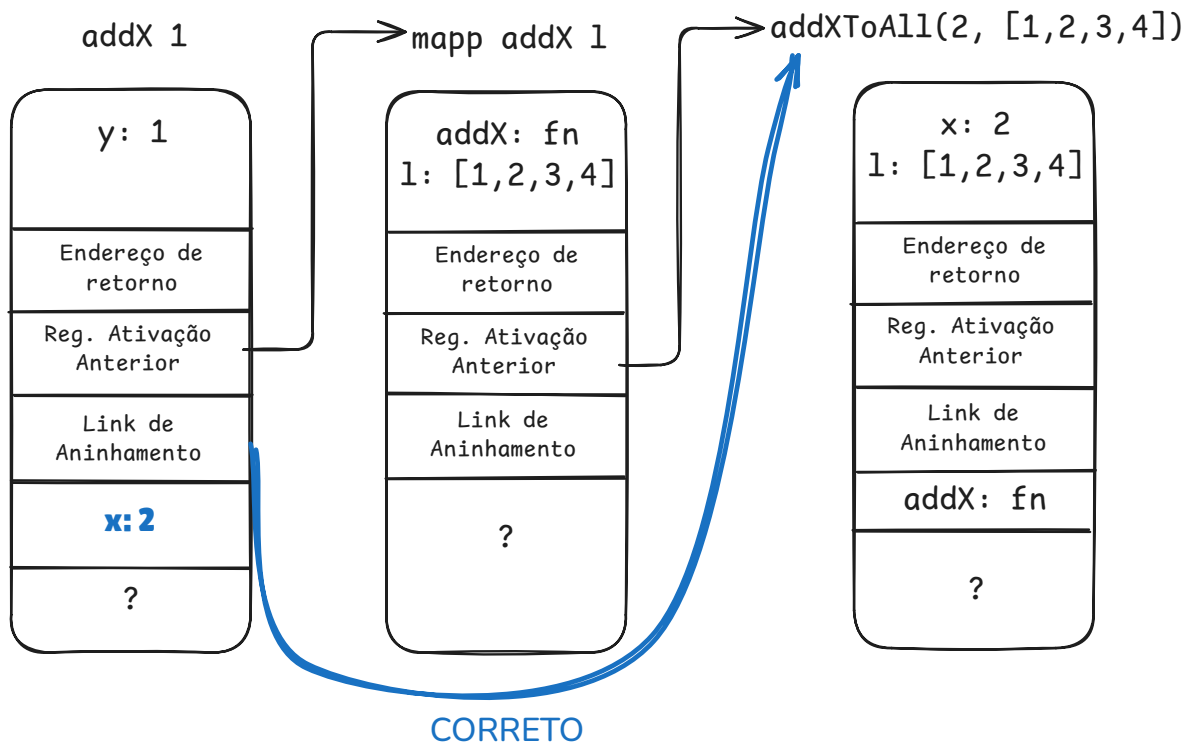
Quais seriam as referências para as ativações das funções?



Como corrigir? 🤔

Quando uma função é passada como parâmetro, **deve-se enviar o LA referente a sua definição**.

Dessa forma, mantém a referência ao escopo de sua definição.



| Funções De Longa Duração

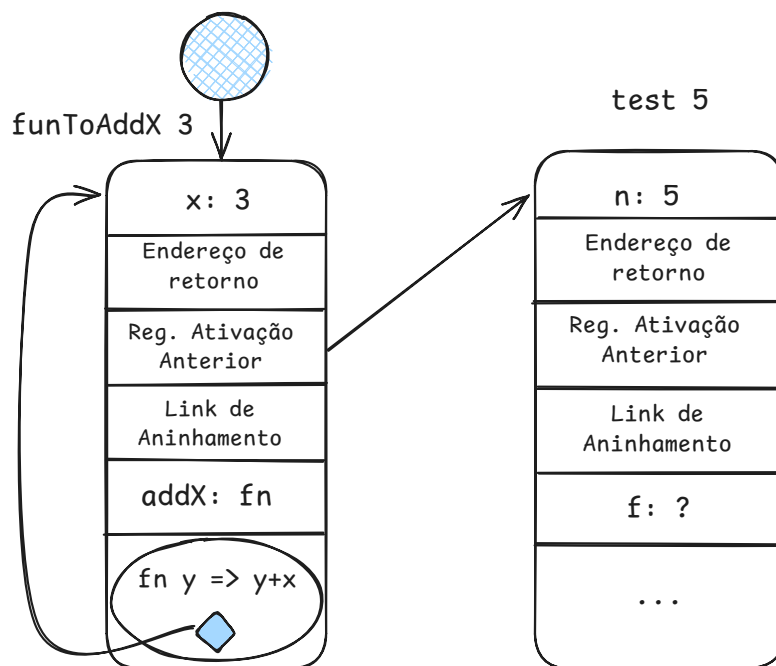
E se o escopo da definição de uma função já não existir mais? 🤔

```
fun funToAddX x =  
  let  
    fun addX y = y + x  
  in  
    addX  
  end;
```

```

fun test n =
  let
    val f = funToAddX 3
  in
    f n
  end;
test 5;

```



4. Coletor de lixo: processo que irá verificar quando um RA pode ser desalocado.